

Estrategia unificada de observabilidad con OpenTelemetry

ADR y pipeline para un e-commerce multinube

[Nombres de los integrantes]

[Institución educativa]

[Asignatura]

[Nombre del docente]

22 de septiembre de 2026

Entrega parcial: puntos 1 y 2 de la actividad

Incluye archivos YAML, diagrama editable y guía de continuidad.

ADR-001. Adoptar OpenTelemetry para la telemetría

Estado: propuesto. Fecha: 22 de septiembre de 2026.

Contexto

La plataforma opera 12 microservicios en GKE y ECS, con 10.000 solicitudes/s. Necesita correlacionar trazas, métricas y logs sin depender de un proveedor. Lenguajes, distribución y spans por solicitud están por confirmar. Se sigue el formato de Nygard (2011).

Opciones consideradas

Opción	Evaluación para este escenario
OpenTelemetry	API y OTLP comunes; permite sustituir destinos. Exige operar Collectors y comprobar compatibilidad.
SDKs específicos del proveedor	Integración directa con funciones nativas; aumenta el trabajo de migración si el código depende de APIs exclusivas.
Agentes propietarios	Despliegue inicial sencillo cuando existe soporte; cobertura, configuración y costes dependen del producto.

Decisión

Adoptar APIs y SDKs OTel, con OTLP a gateways por nube. Instrumentar automáticamente las bibliotecas compatibles y manualmente el negocio. Enviar métricas y logs al backend local y trazas a Jaeger. Se consultó OTel 1.61.0; Collector y convenciones semánticas se versionan por separado (OpenTelemetry, s. f.-e).

Consecuencias

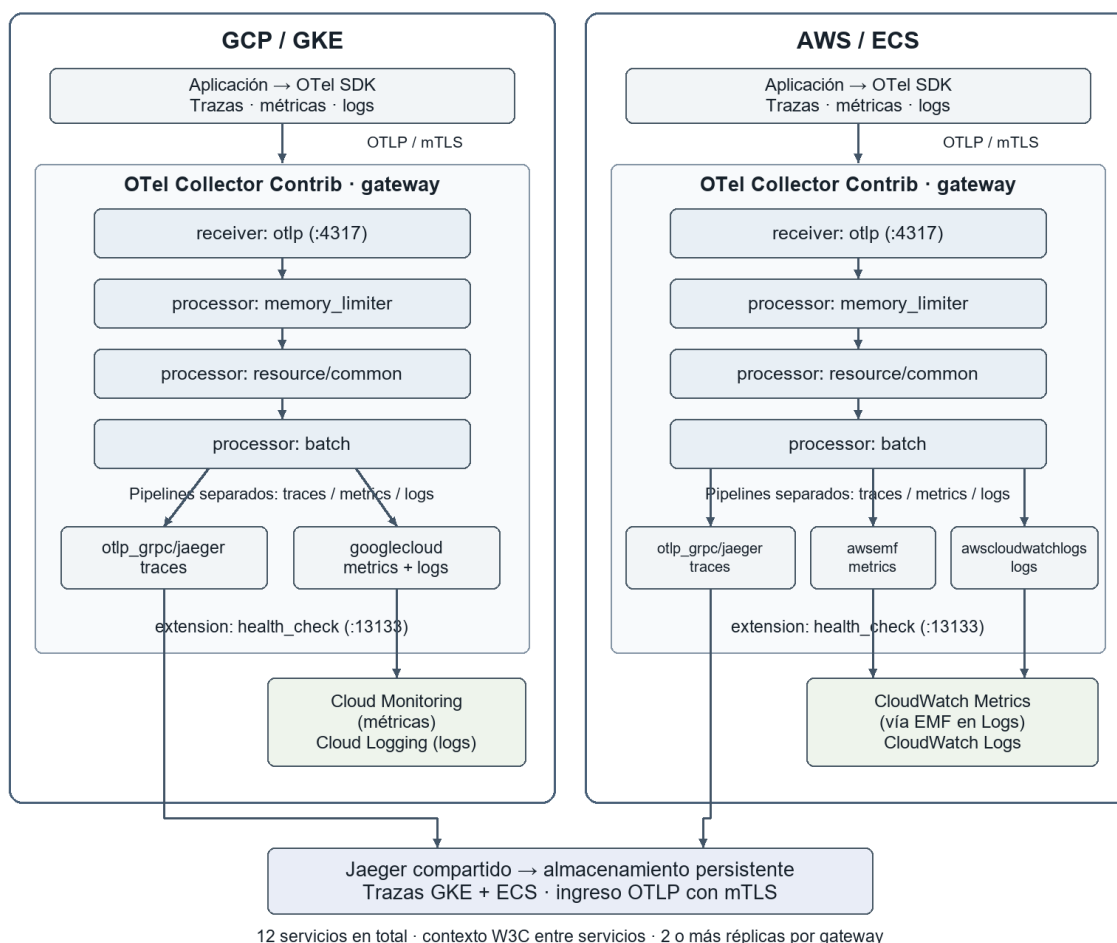
Se reduce el acoplamiento; consultas, alertas e históricos aún requieren migración. CPU, memoria, red y operación añaden costes que deben medirse. Se fija Collector Contrib 0.160.0; su exporter awscloudwatchlogs es alfa y requiere validación (OpenTelemetry, s. f.-b). Sampling y taxonomía quedan para los puntos 4 y 5.

Diseño del pipeline de telemetría

Cada aplicación envía al gateway de su nube: Deployment/Service en GKE y servicio con balanceador interno en ECS. Se parte de dos réplicas por gateway; la capacidad se validará bajo carga.

Figura 1

Pipeline multinube y componentes reales del Collector



Nota. Elaboración propia. Cada pipeline tiene su secuencia de processors; health_check no transporta datos. Jaeger recibe solo trazas (Jaeger, s. f.).

La unificación reside en API, protocolo y contexto. Cloud Logging completa los logs de GCP; awsemf publica métricas vía EMF. Cada señal tiene un destino por nube (OpenTelemetry, s. f.-a, s. f.-d).

Configuración común del Collector

Archivo collector-base.yaml. Se combina con un solo perfil de nube. Los componentes se activan en service.pipelines; definirlos por sí solo no los habilita (OpenTelemetry, s. f.-c).

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      tls:
        cert_file: /etc/otel/tls/server.crt
        key_file: /etc/otel/tls/server.key
        client_ca_file: /etc/otel/tls/ca.crt

processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 768
    spike_limit_mib: 192
  resource/common:
    attributes:
      - key: service.namespace
        value: ecommerce
        action: insert
  batch:
    timeout: 1s
    send_batch_size: 512
    send_batch_max_size: 1024

exporters:
  otlp_grpc/jaeger:
    endpoint: ${env:JAEGER_OTLP_ENDPOINT}
    tls:
      ca_file: /etc/otel/tls/jaeger-ca.crt
      cert_file: /etc/otel/tls/client.crt
      key_file: /etc/otel/tls/client.key
    sending_queue:
      enabled: true
      queue_size: 1000
    retry_on_failure:
      enabled: true
      max_elapsed_time: 300s

extensions:
  health_check:
    endpoint: 0.0.0.0:13133

service:
  extensions: [health_check]
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, resource/common, batch]
      exporters: [otlp_grpc/jaeger]
```

Valores iniciales para ensayo con límite de contenedor de 1 GiB. El límite blando es 576 MiB; batch y la cola consumen memoria adicional. No constituyen un dimensionamiento validado para 10k RPS.

Perfiles de exportación por nube

Se entregan perfiles separados y dos YAML completos ya combinados. Los destinos y las credenciales se resuelven en el despliegue; la aplicación conserva su exportador OTLP.

collector-gcp.yaml

```
exporters:
  googlecloud:
    project: ${env:GCP_PROJECT_ID}
    log:
      default_log_name: ecommerce
service:
  pipelines:
    metrics:
      receivers: [otlp]
      processors:
        - memory_limiter
        - resource/common
        - batch
      exporters: [googlecloud]
    logs:
      receivers: [otlp]
      processors:
        - memory_limiter
        - resource/common
        - batch
      exporters: [googlecloud]
```

collector-aws.yaml

```
exporters:
  awsemf:
    region: ${env:AWS_REGION}
    namespace: Ecommerce
    log_group_name: /ecommerce/metrics
    log_stream_name: ${env:COLLECTOR_ID}
    dimension_rollup_option: NoDimensionRollup
    retain_initial_value_of_delta_metric: true
    resource_to_telemetry_conversion:
      included:
        - service.name
        - deployment.environment.name
    metric_declarations:
      - dimensions:
          - [service.name, deployment.environment.name]
        metric_name_selectors: [".*"]
  awsccloudwatchlogs:
    region: ${env:AWS_REGION}
    log_group_name: /ecommerce/application
    log_stream_name: ${env:COLLECTOR_ID}
service:
  pipelines:
    metrics:
      receivers: [otlp]
      processors:
        - memory_limiter
        - resource/common
        - batch
      exporters: [awsemf]
    logs:
      receivers: [otlp]
      processors:
        - memory_limiter
        - resource/common
        - batch
      exporters: [awsccloudwatchlogs]
```

El SDK debe aportar service.name y deployment.environment.name. Las dimensiones EMF

iniciales se restringen a esos campos; el punto 5 ampliará las dimensiones necesarias para SLIs.

Para AWS se requiere temporalidad delta en sumas e histogramas compatibles, evitando conversiones acumulativas con estado repartido entre réplicas.

Documentación de los exportadores: OpenTelemetry (s. f.-a, s. f.-b, s. f.-d). La ruta de Google utiliza googlecloud por compatibilidad explícita con Cloud Monitoring y Cloud Logging; la instrumentación continúa siendo OTel (Google Cloud, s. f.).

Operación y continuidad del trabajo

Condiciones de despliegue

Montar certificados y configurar Jaeger privado con mTLS y almacenamiento persistente. Definir las variables del perfil. Usar Workload Identity en GKE y un task role en ECS, con permisos de escritura. Restringir 4317 y 13133 a redes internas. La guía adjunta detalla prerequisites.

Cada SDK identifica servicio, versión, instancia y entorno, propaga traceparent/tracestate y conecta el framework de logging. Excluir datos de pago, contraseñas y tokens; evitar duplicar logs mediante OTLP y otro agente.

memory_limiter aplica contrapresión; batch reduce envíos. La cola volátil puede perder datos. Monitorizar memoria, colas y rechazos; probar correlación y caídas de backend. No se ha desplegado ni probado esta propuesta a 10k RPS.

Entrega a los compañeros

Punto pendiente	Trabajo y condición de integración
3. Auto vs. manual	Comparar tiempo de implementación, CPU/memoria y granularidad; medir una carga idéntica con y sin instrumentación.
4. Sampling y SLIs	Justificar la tasa para 10k RPS. Si se elige tail sampling, reunir todos los spans de cada trace_id en el mismo sampler, incluso entre nubes.
5. Semántica	Fijar versión; definir HTTP, DB, RPC y eventos de negocio, unidades, cardinalidad y datos prohibidos.

El YAML reenvía las trazas recibidas; no define sampling ni aprueba retener el 100 % a 10k RPS. Los SLIs usarán métricas de todas las solicitudes elegibles. Muestrear preferentemente errores sesga las estadísticas obtenidas de trazas (OpenTelemetry, s. f.-f).

Referencias

Google Cloud. (s. f.). *Instrument for Cloud Trace*. Recuperado el 22 de septiembre de 2026, de

<https://docs.cloud.google.com/trace/docs/setup>

Jaeger. (s. f.). *APIs*. Recuperado el 22 de septiembre de 2026, de

<https://www.jaegertracing.io/docs/2.21/architecture/apis/>

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect.

<https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>

OpenTelemetry. (s. f.-a). *AWS CloudWatch EMF exporter* (versión 0.160.0) [Código fuente].

GitHub. <https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/v0.160.0/exporter/awsemfexporter>

OpenTelemetry. (s. f.-b). *AWS CloudWatch Logs exporter* (versión 0.160.0) [Código fuente].

GitHub. <https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/v0.160.0/exporter/awsccloudwatchlogsexporter>

OpenTelemetry. (s. f.-c). *Configuration*. Recuperado el 22 de septiembre de 2026, de

<https://opentelemetry.io/docs/collector/configuration/>

OpenTelemetry. (s. f.-d). *Google Cloud exporter* (versión 0.160.0) [Código fuente]. GitHub.

<https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/v0.160.0/exporter/googlecloudexporter>

OpenTelemetry. (s. f.-e). *OpenTelemetry specification 1.61.0*. Recuperado el 22 de septiembre de

2026, de <https://opentelemetry.io/docs/specs/otel/>

OpenTelemetry. (s. f.-f). *Sampling*. Recuperado el 22 de septiembre de 2026, de

<https://opentelemetry.io/docs/concepts/sampling/>